

Phase 3 - Streamlit Dashboard for Local Model Serving

Project context

Phase 3 presents the prediction workflow through a local Streamlit dashboard. The application is designed to consume the best model registered during Phase 2 and to expose the prediction logic through an interactive interface.

Objective

The dashboard should:

- ? load the best model from the MLflow Model Registry
- ? let the user define feature values via sliders and dropdowns
- ? display the predicted petrol price
- ? optionally show a 95 percent confidence interval
- ? present supporting charts and model insights

Implemented application

The current Streamlit application is stored in:

- ? ``phase3/app.py``

It is supported by:

- ? ``phase3/pipeline.py`` as a wrapper to the shared Phase 2 preprocessing logic
- ? ``phase2/model_meta.json`` as metadata source

Implemented dashboard features

User Input Section

The sidebar dynamically creates input controls for:

- ? numeric features such as crude-oil price, tax percentage, year, month, and week of year
- ? categorical features such as country, region, income level, and subsidy level

Prediction Output

The main prediction tab shows:

- ? the predicted fuel price in USD per liter
- ? optional 95 percent confidence interval values

? a compact prediction chart when the OLS confidence interval is available

Key Insights

The dashboard includes additional insight tabs and plots:

- ? feature importance or coefficient overview
- ? target distribution
- ? average petrol price by year

About Section

The application explains:

- ? the dataset
- ? the selected best model
- ? the registry entry
- ? startup instructions
- ? dependency notes

Technical integration

The app expects:

- ? a local MLflow server on `http://localhost:5000`
- ? access to the Phase 2 registry metadata
- ? access to the registered production model

The dashboard is now technically startable and the Streamlit frontend renders correctly.

Important setup note

At the moment, the currently running MLflow server instance does not expose the registered model name `BestFuelRegressor\_v1` on the tested registry endpoint. This means the interface looks correct, but the end-to-end model loading still depends on the MLflow server being started with the same backend store that was used in Phase 2.

In other words:

- ? the Streamlit app structure is correct
- ? the UI is working
- ? the final registry connection still depends on the exact MLflow server configuration

How to run

Terminal 1

Start MLflow locally:

```
c:\Users\maxch\anaconda3\python.exe -m mlflow ui
```

Terminal 2

Start Streamlit:

```
cd c:\Users\maxch\OneDrive\Desktop\HTL\5CHIF\DSAI\MLOPS_Project\phase3
c:\Users\maxch\anaconda3\python.exe -m streamlit run app.py
```

Submission status

Completed

- ? local Streamlit app structure
- ? interactive user input section
- ? prediction panel
- ? optional confidence interval logic
- ? model insight charts
- ? English documentation

Still manual

- ? `phase3.pdf` screenshots of the running dashboard
- ? final screenshot insertion for submission
- ? final verification that the MLflow server points to the correct Phase 2 registry backend

Screenshot placeholders

Recommended screenshots for the final submission:

1. prediction tab with a completed prediction
2. confidence interval output
3. key insights tab
4. about tab

Conclusion

Phase 3 is implemented as a local Streamlit dashboard and fulfills the structural requirements of the assignment. The main remaining task is operational: ensuring that the active MLflow server uses the same backend store as the Phase 2 registry so the production model can be loaded without error.